

EXPERIMENT NO. 7

Create a List of Unique Elements

Programming Language: C

Name : Bodham susma

Register No. : 192571065

Aim

To write a C program that creates a new list containing only the unique elements of a given list of numbers.

Question

Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Test Cases

Some Duplicate Elements

Input: [3, 7, 3, 5, 2, 5, 9, 2]

Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used)

Negative and Positive Numbers

Input: [-1, 2, -1, 3, 2, -2]

Expected Output: [-1, 2, 3, -2] (Order may vary)

List with Large Numbers

Input: [1000000, 999999, 1000000]

Expected Output: [1000000, 999999]

Algorithm

Step 1: Start.

Step 2: Pass the array nums and its size to the function getUniqueElements. If size ≤ 0 , print [] and return.

Step 3: Create an array unique and set uniqueSize = 0.

Step 4: For each element nums[i], check whether it already exists in the unique array.

Step 5: If it is not a duplicate, add it to unique and increment uniqueSize.

Step 6: Print the elements of unique in the form [a, b, c].

Step 7: Stop.

Program (C)

```
#include <stdio.h>

// Function to create a new list with unique elements
void getUniqueElements(int* nums, int size) {
    if (size <= 0) {
        printf("[]\n");
        return;
    }

    int unique[size]; // Array to store the unique elements
    int uniqueSize = 0;

    for (int i = 0; i < size; i++) {
        int isDuplicate = 0;
        // Check if the element already exists in the unique array
        for (int j = 0; j < uniqueSize; j++) {
            if (nums[i] == unique[j]) {
                isDuplicate = 1;
                break;
            }
        }
        // If it's a new element, add it to the unique array
        if (!isDuplicate) {
            unique[uniqueSize] = nums[i];
            uniqueSize++;
        }
    }

    // Print the unique elements list
    printf("[");
    for (int i = 0; i < uniqueSize; i++) {
        printf("%d", unique[i]);
        if (i < uniqueSize - 1) printf(", ");
    }
    printf("]\n");
}

int main() {
    // Test Case 1: Some Duplicate Elements
    int case1[] = {3, 7, 3, 5, 2, 5, 9, 2};
    printf("Test Case 1 -> Expected: [3, 7, 5, 2, 9] | Output: ");
    getUniqueElements(case1, 8);

    // Test Case 2: Negative and Positive Numbers
    int case2[] = {-1, 2, -1, 3, 2, -2};
    printf("Test Case 2 -> Expected: [-1, 2, 3, -2] | Output: ");
    getUniqueElements(case2, 6);

    // Test Case 3: List with Large Numbers
    int case3[] = {1000000, 999999, 1000000};
    printf("Test Case 3 -> Expected: [1000000, 999999] | Output: ");
    getUniqueElements(case3, 3);

    return 0;
}
```

Output

```
Test Case 1 -> Expected: [3, 7, 5, 2, 9] | Output: [3, 7, 5, 2, 9]
Test Case 2 -> Expected: [-1, 2, 3, -2] | Output: [-1, 2, 3, -2]
Test Case 3 -> Expected: [1000000, 999999] | Output: [1000000, 999999]

=== Code Execution Successful ===
```

Complexity Analysis

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

Result

Thus the program was executed successfully and the output was verified.